AOL DATA STRUCTURE KELOMPOK 2 LA95

Kristian Novan – 2802458560

Grace Geidy Christania – 2802479400

Jessica Eileen Handakara – 2802479426

Zahra' Zakiyyah Priyono – 28024492554

For this project, our team developed a student record management system with four main features. The features that our program have are inputting new student data into the database (insert), displaying student data (view). We also has a searching feature to search for a student data using a specific criteria which is ID. Delete feature to delete student data based on ID, as well as feature to exit the program (exit). Handling huge amounts of data efficiently was among our top priorities. Our aim was to design a system such that search and insert operations would be quick and consistent regardless of the size of the data.

We decided to organize the student records using the B-Tree data structure. B-Tree is a self-balancing tree that maintains sorted data and supports efficient search, insert, and delete operations. This structure is quite helpful when dealing with big datasets since it lowers the volume of disc accesses. B-Trees optimized for systems that read and write large blocks of data unlike binary search trees. This made it an ideal choice for our student database system.

Our team selected the B-Tree algorithm mostly for its suitability for search-heavy applications. Because student IDs are unique, which are ideal keys for organizing data in a structured format. For searching, B-Trees provide effective O(log n) time complexity, which helps to quickly find a particular student's record even in large datasets. This is especially helpful in educational settings when data access in real-time is necessary. The B-Tree guarantees that the search will always be fast, regardless of the number of students added.

One other factor determining the choice of B-Trees is their capacity to remain balanced. Adding and deleting student records regularly in a typical academic system might result in an unequal structure in a typical binary search tree. However, B-Trees automatically adjust themselves during insertions and deletions to keep the tree balanced. This balance ensures that all paths from the root to the leaves are of equal length, preventing performance degradation. As a result, operations like insert, delete, and search all perform reliably.

The B-Tree also allows records to be stored in a sorted order based on student ID. This feature supports ordered traversal, which is useful when displaying records in a readable format. The B-Tree structure makes it simple for our group to list all student records by ID. Hash-based systems lack this ordered nature, therefore making B-Trees more flexible for multi-purpose data access. Thus, the use of a B-Tree allows us to efficiently fulfill all functional requirements of the program and provide an optimal user experience.

A. Understanding B-Trees

A B-tree is a tree data structure used to store large amounts of data efficiently and in a structured manner. This structure was introduced to overcome the limitations of binary search trees (BSTs), which tend to become unbalanced when data is added dynamically. In a B-tree, a node can have more than two children and can store multiple keys at once, making it ideal for disk-based storage such as file systems and large databases.

B. Characteristics and Advantages of B-Trees

- All leaf nodes are the same level
- Each node has a minimum and maximum number of keys depending on the degree (t) of the tree
- Searches, insertions, and deletions are performed in O(log n) time
- They can handle large amounts of data without compromising system performance

Another advantage is B-Tree's ability to minimize disk access, as a single node can store multiple data points simultaneously. This makes B-Tree highly suitable for database management systems and search indexes.

C. Use of B-Tree in Student Data Record Management

In its implementation, B-Trees are widely used to build index structures in databases, which are then accessed by applications such as Student Data Record Management. When users whether students or faculty access specific data, the system utilizes the B-Tree index to accelerate searches based on attributes such as student ID numbers as key.

As explained in the HIMMAT BINUS article, the use of data structures like B-Trees can improve the efficiency of search systems in applications involving thousands to millions of data points, such as campus information systems. This structure allows data to be stored in a sequential and indexed manner, making sorting and filtering processes faster.

D. Comparison with Other Structures

Compared to Binary Search Trees (BST) and AVL Trees, B-Trees have advantages in handling large-scale data storage and disk access. BST and AVL are suitable for data in

main memory but are less efficient when working with large storage media because more nodes need to be accessed. Meanwhile, B-Tree reduces tree depth and optimizes data reading from disk, making it the top choice for implementations like Academic Information Systems.

References

• GeeksforGeeks. (n.d.). B-Tree Introduction. Retrieved from https://www.geeksforgeeks.org/b-tree-set-1-introduction-2/

• W3Schools. (n.d.). DSA: B-Trees. Retrieved from https://www.w3schools.com/dsa/dsa_data_btrees.php

• JavaTpoint. (n.d.). B-Tree Data Structure. Retrieved from https://www.javatpoint.com/b-tree

• HIMMAT BINUS. (2022). What is a B-Tree and Its Implementation. Retrieved from https://student-activity.binus.ac.id/himmat/2022/10/apa-itu-b-tree-dan-implementasinya/

Benefit Using B-Tree:

A. Efficient for large data on disk

B-Trees are meant to effectively manage great volumes of data kept on disk. Storing several keys in a single node, B-Trees reduce disc reads unlike binary search trees that might need regular memory access. This is especially important when using slow storage devices as it lowers the number of accesses needed to find a piece of data. B-Trees preserve their performance as data increases by keeping logarithmic complexity for actions like insertion, removal, and search. It is ideal for uses requiring large datasets, such student records in educational institutions.

B. Fast search time

The capacity of B-Trees to conduct quick searches even in large datasets is among their prominent features. By maintaining a sorted structure and allowing multiple keys per node, B-Trees significantly reduce the depth of the tree, which minimizes the number of steps needed to find a specific value. It is crucial especially in systems where user experience depends on search speed as in student ID searches. Search operations remain always quick even as the number of records grows. This means that for systems needing regular and fast data access, B-Trees provide an excellent solution.

C. Balanced tree

B-Trees are inherently balanced, which means all paths from the root to the leaf nodes are of equal length. This balance ensures that no part of the tree becomes disproportionately deep, preventing performance loss over time. Maintaining balance automatically during insertions and deletions keeps operations predictable and efficient. For uses such as databases where unequal tree growth might cause access times to be inconsistent, this structural feature is essential. B-Trees enable developers to count on consistent performance regardless of the order or frequency of data operations.

D. Supports multiple data items per node

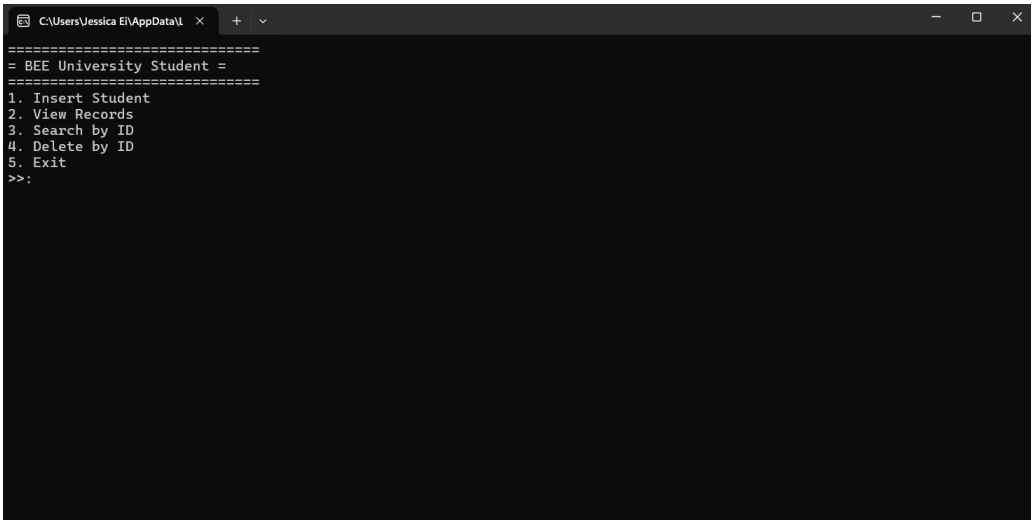
Unlike binary trees, B-Trees allow each node to contain more than one key and child pointer. With this ability, the tree lowers its total height, which then decrease the number of node visits during search and update activities. Having multiple data items per node also improves cache performance and disk read efficiency. For disk-based storage systems' performance, it lets the system handle more data with less I/O operations. This feature makes B-Trees particularly suitable for environments where memory and speed optimization are priorities.

E. Suitable for databases and file systems

B-Trees are the standard data structure used in many database and file system implementations due to their combination of speed, balance, and scalability. Their ability to maintain sorted data and perform range queries efficiently is important for indexing in relational databases. B-Trees also help file systems by arranging directory structures and file indexes to enable rapid access and updates. The design of B-Trees fits naturally with the needs of persistent storage systems that must manage large, dynamic sets of data reliably.

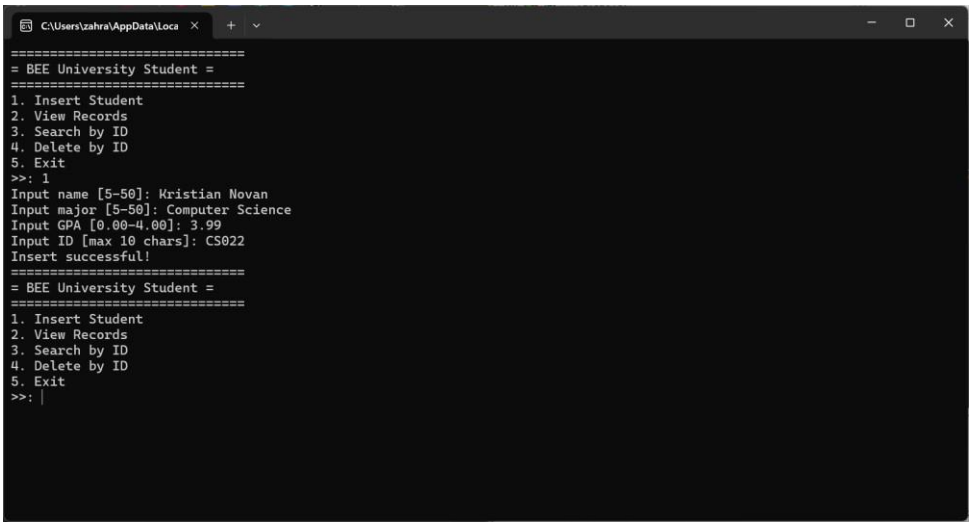
All Feature Screenshot

A. Main Menu

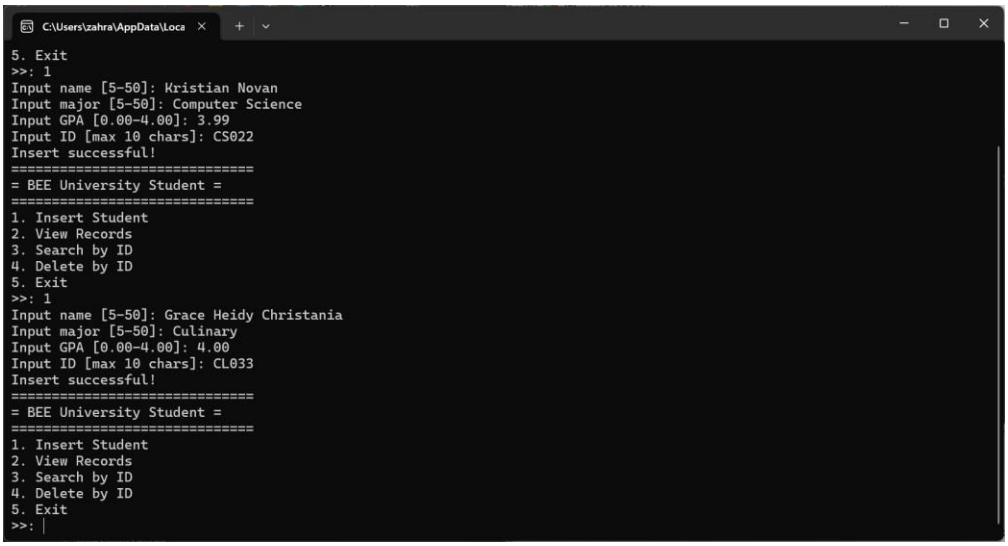


B. Insert Student

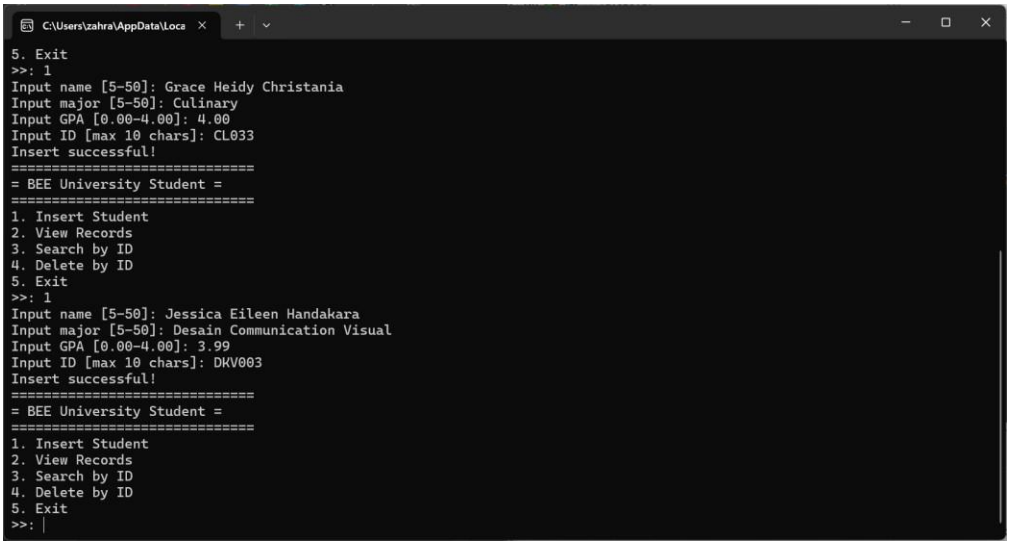
1. Input Kristian Novan



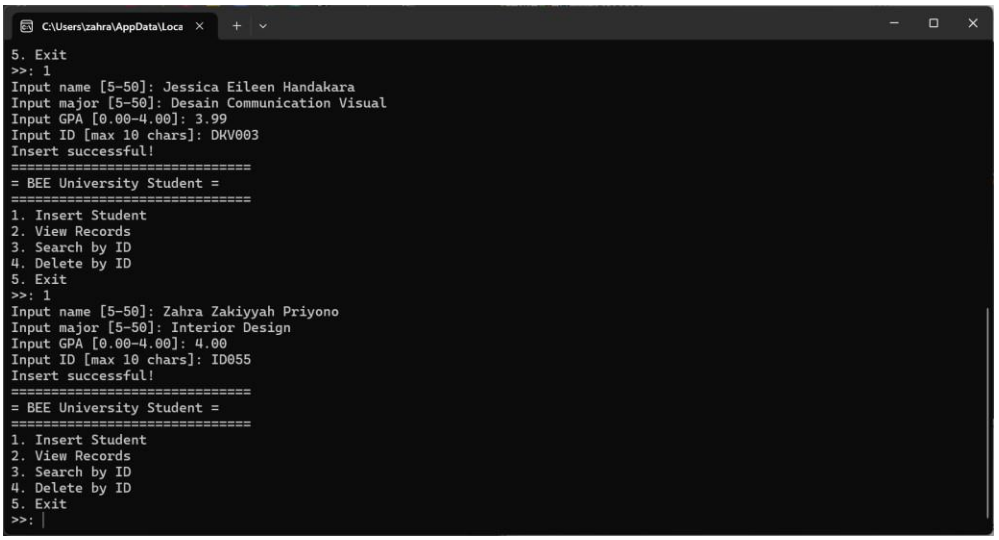
2. Input Grace Heidy Christania



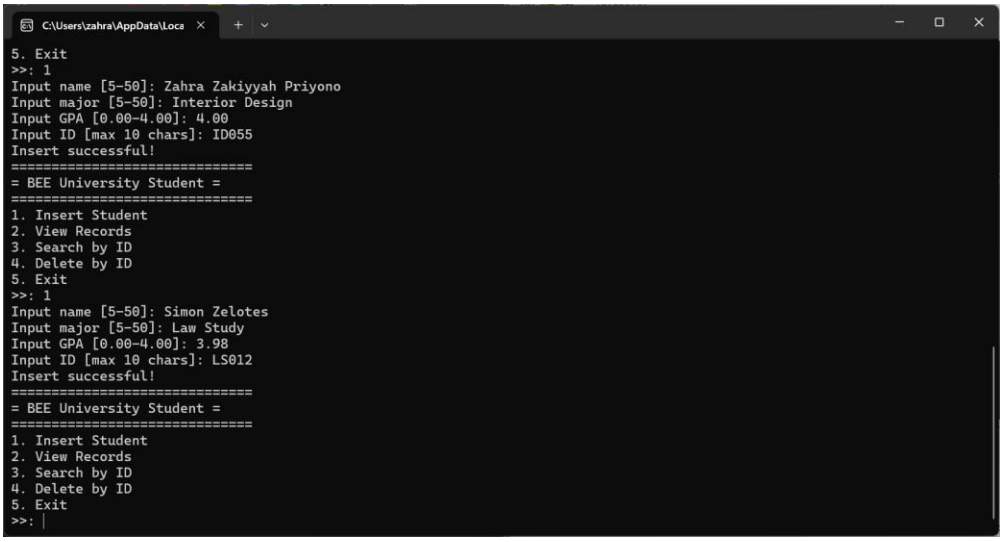
3. Input Jessica Eileen Handakara



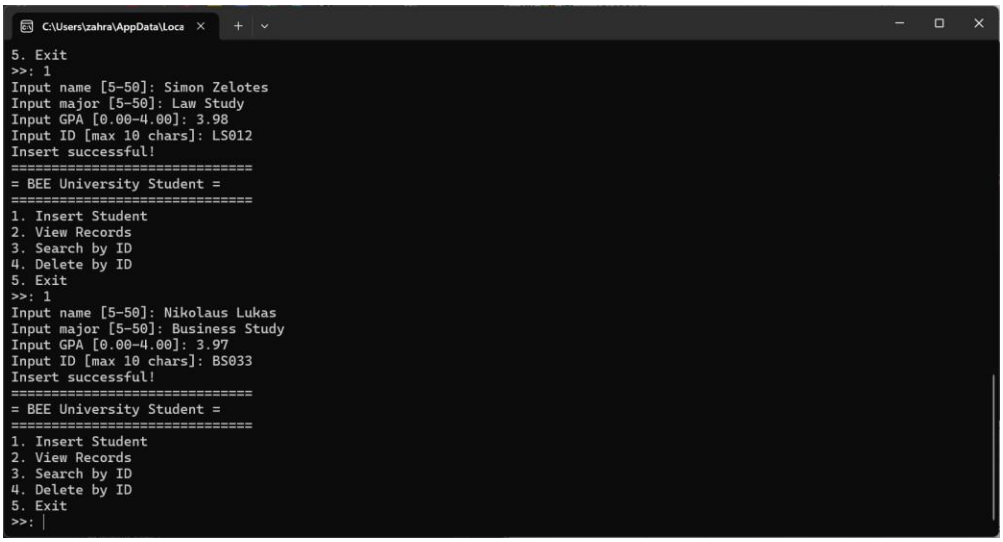
4. Input Zahra' Zakiyyah Priyono



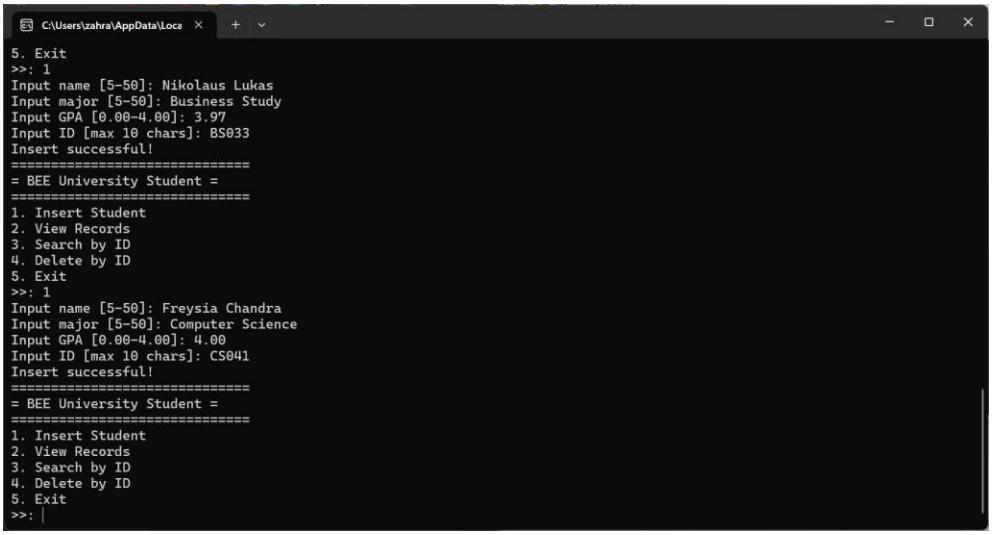
5. Input Simon Zelotes



6. Input Nikolaus Lukas

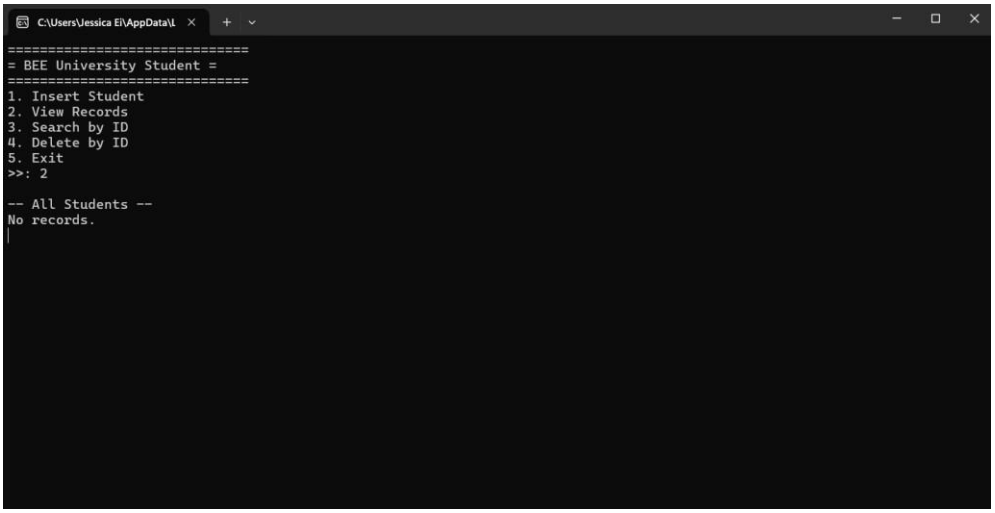


7. Input Freysia Chandra

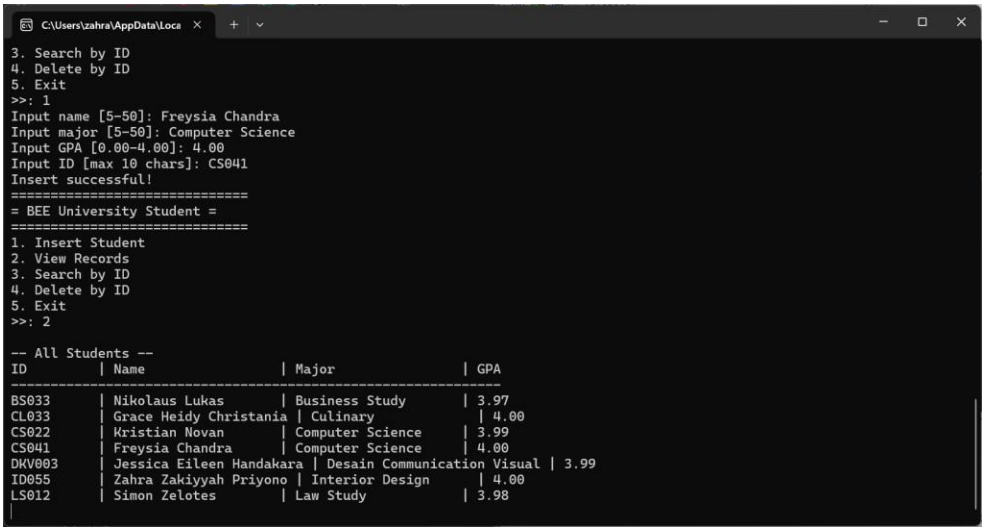


C. View Records

1. If there is no input before

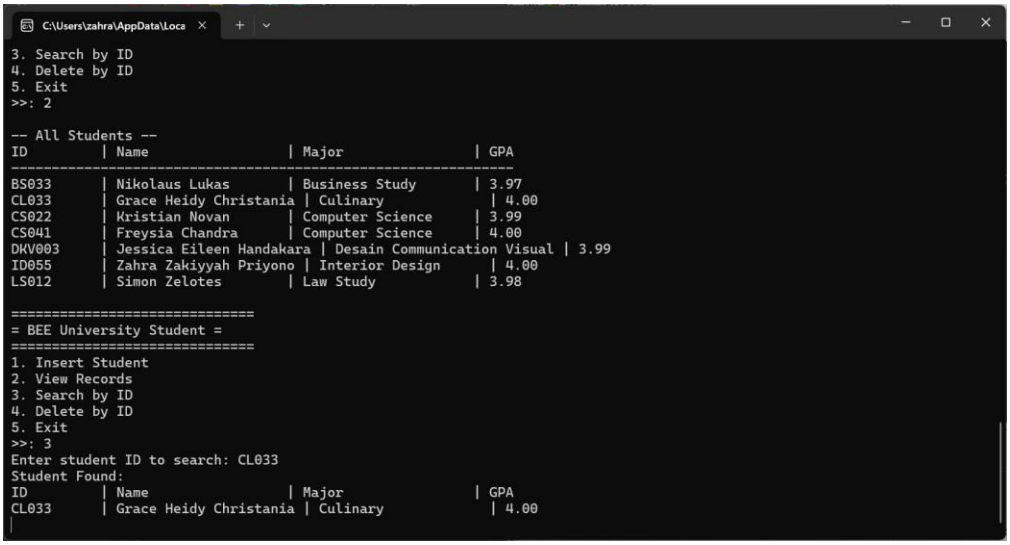


2. After inputing data



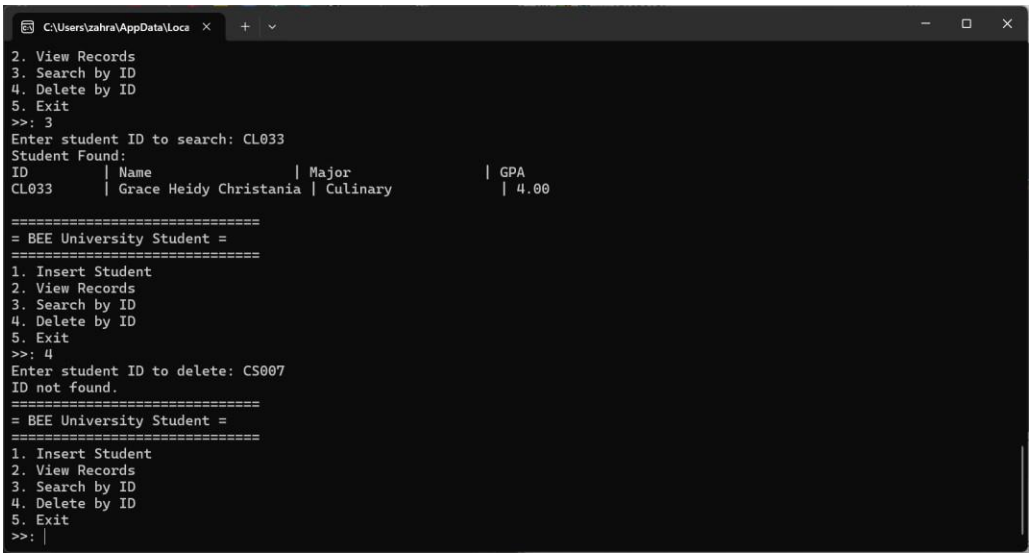
D. Search by ID

Search ID “CL033”

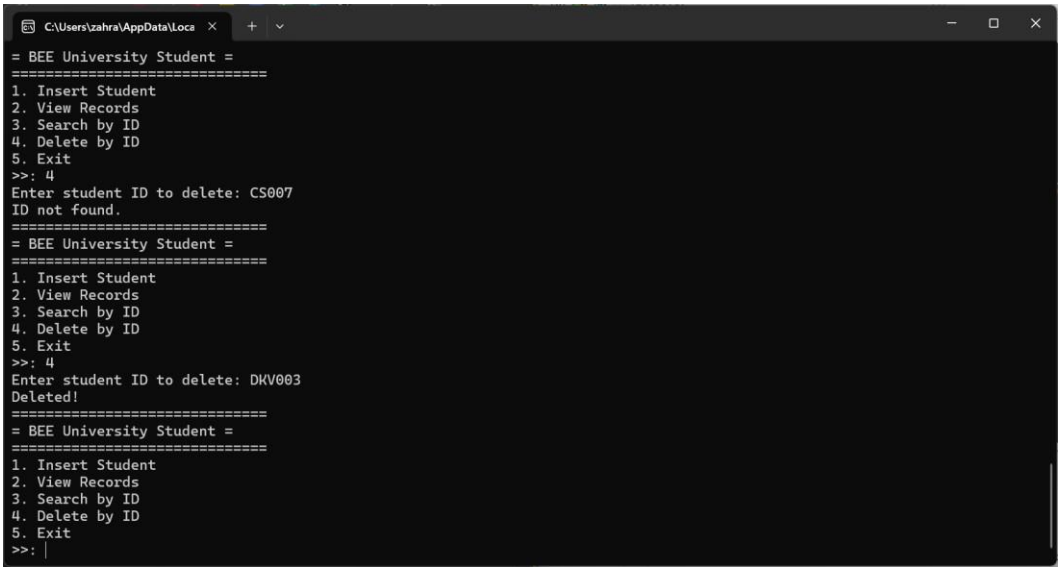


E. Delete by ID

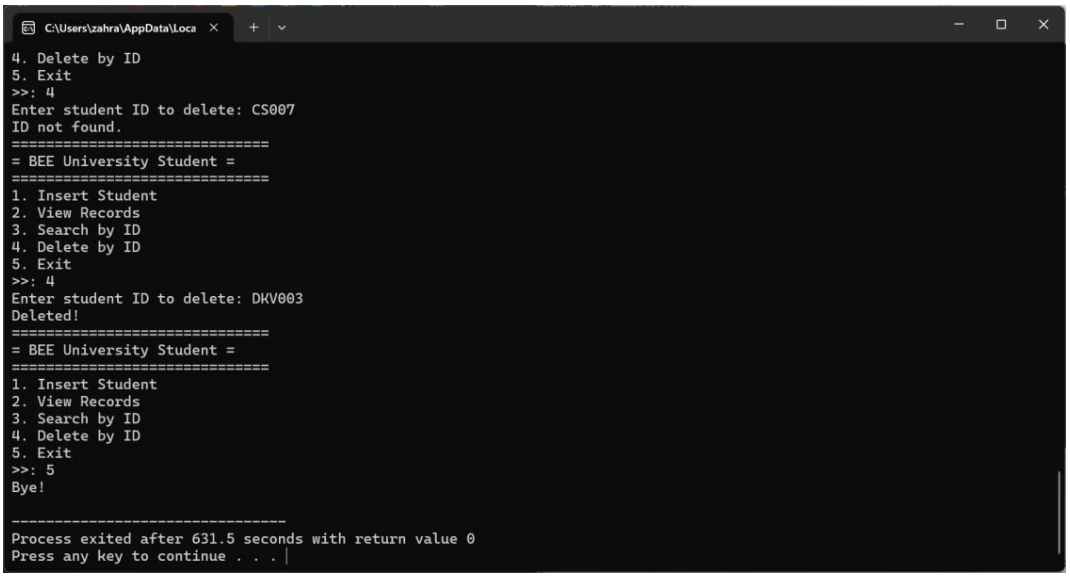
1. Delete by ID if ID is not in the data. Delete “CS007”



2. Delete by ID if ID is in the data



F. Exit


C:\Users\zahra\AppData\Loca
4. Delete by ID
5. Exit
>>: 4
Enter student ID to delete: CS007
ID not found.
==============================
= BEE University Student =
==============================
1. Insert Student
2. View Records
3. Search by ID
4. Delete by ID
5. Exit
>>: 4
Enter student ID to delete: DKV003
Deleted!
==============================
= BEE University Student =
==============================
1. Insert Student
2. View Records
3. Search by ID
4. Delete by ID
5. Exit
>>: 5
Bye!

Process exited after 631.5 seconds with return value 0
Press any key to continue . . .

Coding Documentation (with Human Language)

You can check // for the explanation

```c
#include <stdio.h> // library yang digunakan
#include <stdlib.h> // library yang digunakan
#include <string.h> // library yang digunakan
#include <stdbool.h> // library yang digunakan

#define MIN_DEGREE 3 // ini merukanan nilai minimum (t) dari B-Tree
// rumus node punya maksimal (2t - 1)key
// rumus node punya minimal (t - 1)key

typedef struct StudentRecord { // ini merupakan stuktur data yang menyimpan data mahasiswa
    char studentID[11];
    char name[52];
    char major[52];
    float gpa;
} StudentRecord;
```

```c

typedef struct BTreeNode { // ini untk B-Tree node nya
    int n; // jumblah key dalam node
    char (*keys)[10]; // ini array key
    StudentRecord *records; // ini array data mahasiswa
    struct BTreeNode **children; // array pointer ke children
    bool isLeaf; // check apakah ini leaf
    int t; // degree B-Tree
} BTreeNode;

typedef struct BTree {
    BTreeNode *root;
    int t;
    int id_length;
} BTree;

BTree *studentTree;
```

```c

// ini untuk membuat node baru
BTreeNode* createNode(int t_val, bool leaf_val) {
    BTreeNode* newNode =  (BTreeNode*) malloc(sizeof(BTreeNode));
    newNode->t = t_val;
    newNode->isLeaf = leaf_val;
    newNode->n = 0;
    newNode->keys = (char (*)[10])malloc(sizeof(char[10]) * (2 * t_val - 1));
    newNode->records = (StudentRecord*) malloc(sizeof(StudentRecord) * (2 * t_val - 1));
    newNode->children = (BTreeNode**) malloc(sizeof(BTreeNode*) * (2 * t_val));
    return newNode;
}

// ini untuk inalisasi B-Tree
BTree* createBTree(int t_val, int id_len) {
    BTree* tree = (BTree*)malloc(sizeof(BTree));
    tree->root = NULL;
    tree->t = t_val;
    tree->id_length = id_len;
    return tree;
}
```

```
// fungsi ini untuk mencari indeks untuk key
int findKeyIndex(BTreeNode *node, const char *key) {
    int idx = 0;
    while (idx < node->n && strcmp(node->keys[idx], key) < 0) idx++;
    return idx;
}

// untuk mendapatkan predecessor
char* getPred(BTreeNode *node, int idx) {
    BTreeNode *cur = node->children[idx];
    while (!cur->isLeaf) cur = cur->children[cur->n];
    return cur->keys[cur->n - 1];
}
// untuk mendapatkan successor
char* getSucc(BTreeNode *node, int idx) {
    BTreeNode *cur = node->children[idx + 1];
    while (!cur->isLeaf) cur = cur->children[0];
    return cur->keys[0];
}
```

```
// fungsi untuk meminjam 1 data dari siblings kiri ke child yang kekurangan data
// saat node dalam B-Tree kekurangan isi
void borrowFromPrev(BTreeNode *node, int idx) {
    BTreeNode *child = node->children[idx];
    BTreeNode *sib = node->children[idx - 1];

    for (int i = child->n - 1; i >= 0; i--) {
        strcpy(child->keys[i + 1], child->keys[i]);
        child->records[i + 1] = child->records[i];
    }
    if (!child->isLeaf) {
        for (int i = child->n; i >= 0; i--)
            child->children[i + 1] = child->children[i];
    }
    strcpy(child->keys[0], node->keys[idx - 1]);
    child->records[0] = node->records[idx - 1];
    if (!child->isLeaf)
        child->children[0] = sib->children[sib->n];
    strcpy(node->keys[idx - 1], sib->keys[sib->n - 1]);
    node->records[idx - 1] = sib->records[sib->n - 1];
    child->n++;
    sib->n--;
}
```

```c
// fungsi untuk meminjam 1 data dari siblings kanan ke child yang kekurangan data
// saat node dalam B-Tree kekurangan isi setelah penghapusan data
void borrowFromNext(BTreeNode *node, int idx) {
    BTreeNode *child = node->children[idx];
    BTreeNode *sib = node->children[idx + 1];

    strcpy(child->keys[child->n], node->keys[idx]);
    child->records[child->n] = node->records[idx];
    if (!child->isLeaf)
        child->children[child->n + 1] = sib->children[0];

    strcpy(node->keys[idx], sib->keys[0]);
    node->records[idx] = sib->records[0];

    for (int i = 1; i < sib->n; i++) {
        strcpy(sib->keys[i - 1], sib->keys[i]);
        sib->records[i - 1] = sib->records[i];
    }
    if (!sib->isLeaf) {
        for (int i = 1; i <= sib->n; i++)
            sib->children[i - 1] = sib->children[i];
    }
    child->n++;
    sib->n--;
}
```

```c
// digunakan untuk gabungin dua child yang isinya dikit jadi satu anak yang lebih penuh
// saat penghapusan data di B-Tree
void mergeChildren(BTreeNode *node, int idx) {
    BTreeNode *child = node->children[idx];
    BTreeNode *sib = node->children[idx + 1];

    strcpy(child->keys[child->n], node->keys[idx]);
    child->records[child->n] = node->records[idx];
    child->n++;
    for (int i = 0; i < sib->n; i++) {
        strcpy(child->keys[child->n + i], sib->keys[i]);
        child->records[child->n + i] = sib->records[i];
    }
    if (!child->isLeaf) {
        for (int i = 0; i <= sib->n; i++)
            child->children[child->n + i] = sib->children[i];
    }
    child->n += sib->n;

    for (int i = idx + 1; i < node->n; i++) {
        strcpy(node->keys[i - 1], node->keys[i]);
        node->records[i - 1] = node->records[i];
    }
    for (int i = idx + 2; i <= node->n; i++)
        node->children[i - 1] = node->children[i];
    node->n--;
    free(sib->keys);
    free(sib->records);
    free(sib->children);
    free(sib);
}
```

```
156  // memastikan child memiliki cukup key
157  void fillChild(BTreeNode *node, int idx) {
158      if (idx && node->children[idx - 1]->n >= node->t)
159          borrowFromPrev(node, idx);
160      else if (idx != node->n && node->children[idx + 1]->n >= node->t)
161          borrowFromNext(node, idx);
162      else {
163          if (idx != node->n) mergeChildren(node, idx);
164          else mergeChildren(node, idx - 1);
165      }
166  }
167
168  // buat nyari 1 data mahasiswa berdasaryakn key
169  StudentRecord* search_node(BTreeNode *node, const char *key) {
170      int i = 0;
171      while (i < node->n && strcmp(key, node->keys[i]) > 0) i++;
172      if (i < node->n && strcmp(key, node->keys[i]) == 0)
173          return &node->records[i];
174      if (node->isLeaf) return NULL;
175      return search_node(node->children[i], key);
176  }
177
178  // tambahan buat cek pohonnya ada atau engga
179  StudentRecord* search_student(BTree *tree, const char *key) {
180      if (!tree->root) return NULL;
181      return search_node(tree->root, key);
182  }
183
184  // menampilkan seluruh data yang ada di dalam B-Tree secara berurutan
185  void traverse(BTreeNode *node) {
186      int i;
187      for (i = 0; i < node->n; i++) {
188          if (!node->isLeaf) traverse(node->children[i]);
189          printf("%-10s | %-20s | %-20s | %.2f\n",
190                  node->records[i].studentID,
191                  node->records[i].name,
192                  node->records[i].major,
193                  node->records[i].gpa);
194      }
195      if (!node->isLeaf) traverse(node->children[i]);
196  }
197
```

```
197
198 // fungsi kalau node di B-Tree sudah penuh, dan kita mau masukin data baru
199 // kita bagi node menjadi dua child
200 void splitChild(BTreeNode *parent, int idx, BTreeNode *child) {
201     BTreeNode *newChild = createNode(child->t, child->isLeaf);
202     newChild->n = child->t - 1;
203
204     for (int j = 0; j < child->t - 1; j++) {
205         strcpy(newChild->keys[j], child->keys[j + child->t]);
206         newChild->records[j] = child->records[j + child->t];
207     }
208     if (!child->isLeaf) {
209         for (int j = 0; j < child->t; j++)
210             newChild->children[j] = child->children[j + child->t];
211     }
212     child->n = child->t - 1;
213     for (int j = parent->n; j >= idx + 1; j--)
214         parent->children[j + 1] = parent->children[j];
215     parent->children[idx + 1] = newChild;
216     for (int j = parent->n - 1; j >= idx; j--) {
217         strcpy(parent->keys[j + 1], parent->keys[j]);
218         parent->records[j + 1] = parent->records[j];
219     }
220     strcpy(parent->keys[idx], child->keys[child->t - 1]);
221     parent->records[idx] = child->records[child->t - 1];
222     parent->n++;
223 }
```

```
225 // memasukkan data baru ke dalam node B-Tree yang belum penuh
226 void insertNonFull(BTreeNode *node, const char *key, StudentRecord rec) {
227     int i = node->n - 1;
228     if (node->isLeaf) {
229         while (i >= 0 && strcmp(node->keys[i], key) > 0) {
230             strcpy(node->keys[i + 1], node->keys[i]);
231             node->records[i + 1] = node->records[i];
232             i--;
233         }
234         strcpy(node->keys[i + 1], key);
235         node->records[i + 1] = rec;
236         node->n++;
237     } else {
238         while (i >= 0 && strcmp(node->keys[i], key) > 0) i--;
239         i++;
240         if (node->children[i]->n == 2 * node->t - 1) {
241             splitChild(node, i, node->children[i]);
242             if (strcmp(node->keys[i], key) < 0) i++;
243         }
244         insertNonFull(node->children[i], key, rec);
245     }
246 }
```

```
248  // memasukkan data mahasiswa baru ke dalam B-Tree
249  void insert_student(BTree *tree, const char *key, const char *name,
250                      const char *major, float gpa) {
251      StudentRecord rec;
252      strncpy(rec.studentID, key, 10);
253      strncpy(rec.name, name, 50);
254      strncpy(rec.major, major, 50);
255      rec.gpa = gpa;
256
257      if (!tree->root) {
258          tree->root = createNode(tree->t, true);
259          strcpy(tree->root->keys[0], key);
260          tree->root->records[0] = rec;
261          tree->root->n = 1;
262      } else {
263          if (tree->root->n == 2 * tree->t - 1) {
264              BTreeNode *s = createNode(tree->t, false);
265              s->children[0] = tree->root;
266              splitChild(s, 0, tree->root);
267              int idx = strcmp(s->keys[0], key) < 0 ? 1 : 0;
268              insertNonFull(s->children[idx], key, rec);
269              tree->root = s;
270          } else {
271              insertNonFull(tree->root, key, rec);
272          }
273      }
274  }
```

```c
// menghapus sebuah data (key) dari node B-Tree
void deleteFromNode(BTreeNode *node, const char *key) {
    int idx = findKeyIndex(node, key);

    if (idx < node->n && strcmp(node->keys[idx], key) == 0) {
        if (node->isLeaf) {
            for (int i = idx + 1; i < node->n; i++) {
                strcpy(node->keys[i - 1], node->keys[i]);
                node->records[i - 1] = node->records[i];
            }
            node->n--;
        } else {
            if (node->children[idx]->n >= node->t) {
                char *predKey = getPred(node, idx);
                StudentRecord predRec = *search_node(node->children[idx], predKey);
                strcpy(node->keys[idx], predKey);
                node->records[idx] = predRec;
                deleteFromNode(node->children[idx], predKey);
            }
            else if (node->children[idx + 1]->n >= node->t) {
                char *succKey = getSucc(node, idx);
                StudentRecord succRec = *search_node(node->children[idx + 1], succKey);
                strcpy(node->keys[idx], succKey);
                node->records[idx] = succRec;
                deleteFromNode(node->children[idx + 1], succKey);
            } else {
                mergeChildren(node, idx);
                deleteFromNode(node->children[idx], key);
            }
        }
    }
    else {
        if (node->isLeaf) return;
        bool flag = (idx == node->n);
        if (node->children[idx]->n < node->t)
            fillChild(node, idx);

        if (flag && idx > node->n)
            deleteFromNode(node->children[idx - 1], key);
        else
            deleteFromNode(node->children[idx], key);
    }
}
```

```c
// untuk menjaga agar struktur pohon tetap benar dan konsisten setelah penghapusan data
void deleteKey(BTree *tree, const char *key) {
    if (!tree->root) return;
    deleteFromNode(tree->root, key);

    if (tree->root->n == 0) {
        BTreeNode *tmp = tree->root;
        if (tmp->isLeaf) {
            free(tmp->keys);
            free(tmp->records);
            free(tmp->children);
            free(tmp);
            tree->root = NULL;
        } else {
            tree->root = tmp->children[0];
            free(tmp->keys);
            free(tmp->records);
            free(tmp->children);
            free(tmp);
        }
    }
}
```

```c
// untuk menu insert data mahasiswa ke dalam B-Tree
void menu_insert() {
    char name[52], major[52], id_input[11];
     float gpa;

     printf("Input name [5-50]: ");
     scanf(" %[^\n]", name);
     while (strlen(name) < 5 || strlen(name) > 50) {
         printf("Invalid name. Try again: ");
         scanf(" %[^\n]", name);
     }

     printf("Input major [5-50]: ");
     scanf(" %[^\n]", major);
     while (strlen(major) < 5 || strlen(major) > 50) {
         printf("Invalid major. Try again: ");
         scanf(" %[^\n]", major);
     }

     printf("Input GPA [0.00-4.00]: ");
     scanf("%f", &gpa);
     while (gpa < 0.00 || gpa > 4.00) {
         printf("Invalid GPA. Try again: ");
         scanf("%f", &gpa);
     }

     printf("Input ID [max 10 chars]: ");
     scanf(" %[^\n]", id_input);
     while (strlen(id_input) >= 11 || search_student(studentTree, id_input)) {
         printf("ID invalid or exists. Try again: ");
         scanf(" %[^\n]", id_input);
     }

     insert_student(studentTree, id_input, name, major, gpa);
     printf("Insert successful!\n");
     getchar();
}
```

```c
// ini untuk view
void menu_view() {
    printf("\n-- All Students --\n");
    if (!studentTree->root) {
        printf("No records.\n");
    } else {
        printf("%-10s | %-20s | %-20s | %s\n", "ID", "Name", "Major", "GPA");
        printf("-------------------------------------------------------------\n");
        traverse(studentTree->root);
    }
    getchar(); getchar();
}
```

```c
// ini untuk delete
void menu_delete() {
    char id[11];
    printf("Enter student ID to delete: ");
    scanf(" %10[^\n]", id);
    if (!search_student(studentTree, id)) {
        printf("ID not found.\n");
    } else {
        deleteKey(studentTree, id);
        printf("Deleted!\n");
    }
}

// ini main menu untuk menjalankan program
int main() {
    studentTree = createBTree(MIN_DEGREE, 10);
    int choice;
    do {
        printf("==============================\n");
        printf("= BEE University Student =\n");
        printf("==============================\n");
        printf("1. Insert Student\n");
        printf("2. View Records\n");
        printf("3. Search by ID\n");
        printf("4. Delete by ID\n");
        printf("5. Exit\n");
        printf(">>: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: menu_insert(); break;
            case 2: menu_view(); break;
            case 3: menu_search(); break;
            case 4: menu_delete(); break;
            case 5: printf("Bye!\n"); break;
            default: printf("Invalid\n");
        }
    } while (choice != 5);
    return 0;
}
```

```c
// ini untuk search
void menu_search() {
    char id_to_search[11];
    printf("Enter student ID to search: ");
    scanf(" %[^\n]", id_to_search);

    StudentRecord* result = search_student(studentTree, id_to_search);
    if (result) {
        printf("Student Found:\n");
        printf("%-10s | %-20s | %-20s | %s\n", "ID", "Name", "Major", "GPA");
        printf("%-10s | %-20s | %-20s | %.2f\n",
               result->studentID,
               result->name,
               result->major,
               result->gpa);
    } else {
        printf("Student ID %s not found.\n", id_to_search);
    }

    getchar(); getchar();
}
```